Full-stack software development

Lab 02: React and Next.js introduction

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Contents

1 Introduction

In this lab, you will learn the fundamentals of React by building a dynamic and interactive web application. We will create a page that displays an overview of lecturers and, upon selecting a lecturer, shows the courses they teach.

The concepts covered include:

- JSX: The syntax used to write HTML-like code in React components.
- React Components: The building blocks of a React application.
- Properties: How to pass data from a parent to a child component.
- State: How to give components their own "memory" to manage interactive data.
- Callback Functions: A pattern for child components to communicate back to their parents.
- Conditional Rendering: How to show or hide components based on the application's state.

In addition to the presentation slides, you can use the links below to successfully complete this lab:

- https://react.dev/reference/react
- https://nextjs.org/docs

2 Getting started

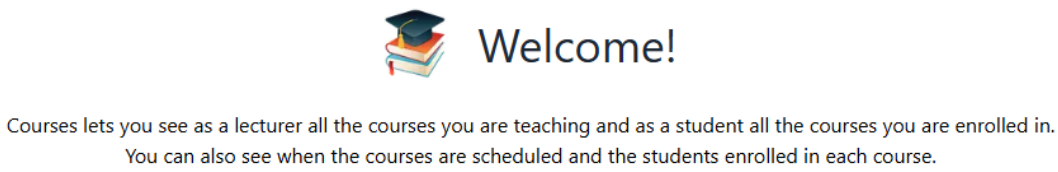
To begin this lab, please accept the following GitHub assignment:
https://classroom.github.com/a/JFK_xAY3.

Once you have accepted the assignment, a repository containing the start code will be created for you. Clone this repository to a local directory and open it in VSCode.
Next, ensure all the necessary dependencies for the project are installed. Open a terminal in the root directory of the project and run the following command:

In a second terminal, install and start the front-end:

```
npm install
npm run dev
```

In your browser, navigate to http://localhost:8080, which should look like this:



3 Creating a shared layout

Navigate between the home page at http://localhost:8080 and the lecturers page at http://localhost:8080/lecturers. You'll notice a clear lack of consistency. While both pages are still quite bare, the main issue is that there is no shared header or navigation. Let's fix this first.

In Next.js, the file app/layout.tsx serves as the main template for the entire application. Any component you add here will be displayed on every page. Open the app/layout.tsx file. You will see the following structure:

```
const RootLayout = async ({ children }: { children: React.ReactNode })
=> {
  return (
    <html lang="en">
      <body>
        <main className="p-6 min-h-screen flex flex-col items-center">
          {children}
        </main>
      </body>
    </html>
  );
};
export default RootLayout;
```

The *{children}* prop is a placeholder where the content of the current page (like app/page.tsx or app/lecturers/page.tsx) will be rendered.

Let's add the Header component, which has already been created for you, to this root layout.

1. Import the Header component at the top of app/layout.tsx:

```
import Header from '@components/header';
```

2. Add the <Header /> component inside the <body> tag, just before the <main> tag:

```
<Header />
```

Save the file and check your browser. The header with navigation links is now visible on both the home page and the lecturers page. Any new page you create will automatically share this same layout.

4 Building the Lecturer Overview

Our goal for this section is to connect our data to our UI. We will fetch a list of lecturers and display them in a table.

4.1 Add the LecturerOverviewTable component

1) Edit the parent component, app/lecturers/page.tsx. This component will be responsible for fetching the data and passing it down to its child. Replace the static *<section>Lecturer Overview</section>* with our LecturerOverviewTable component:

```
<section>
    <LecturerOverviewTable lecturers={data} />
</section>
```

2) Import this component in the page.
3) Look at the code for the LecturerOverviewTable, you'll notice it has a prop called lecturers. This prop (or parameter) will be used in the next step to dynamically render the lecturers in table rows. This code won't compile yet, as we need to fetch the data.

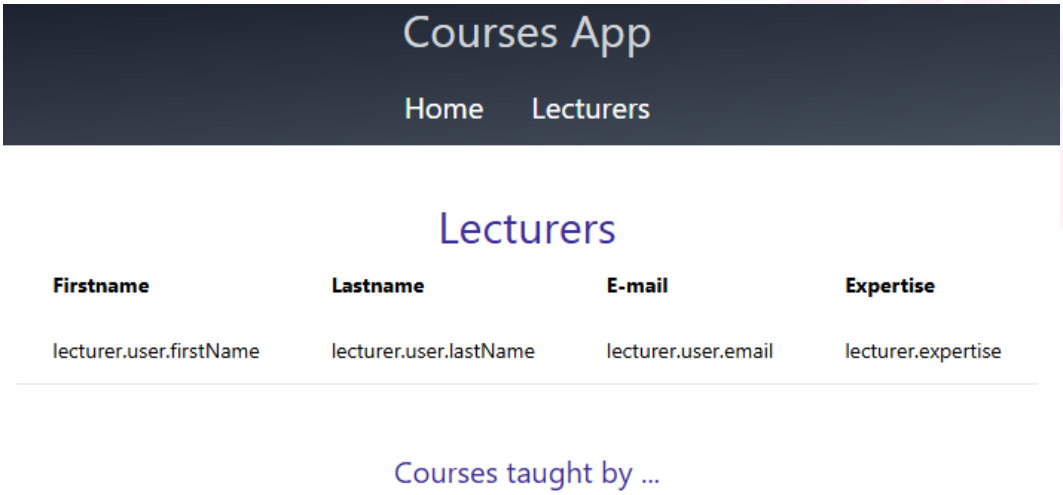
4.2 Fetch data via Services

1) Have a look at *services/LecturerService.ts.* For now, it returns a hard-coded list of data, so we are not yet calling a real backend API. Later, we can easily change the service to fetch data from a real API without having to modify any of the components that use it.

In a well-structured application, we separate the data-fetching logic from the presentation logic (our components). By encapsulating this logic in a service, components can simply call these services without needing to know how or where the data comes from.

2) Implement the getData function to fetch our lecturer data. This function needs to call our LecturerService.

3) Call the getData function in our LecturersPage component.
4) In the browser, navigate to /lecturers. This should look like this:



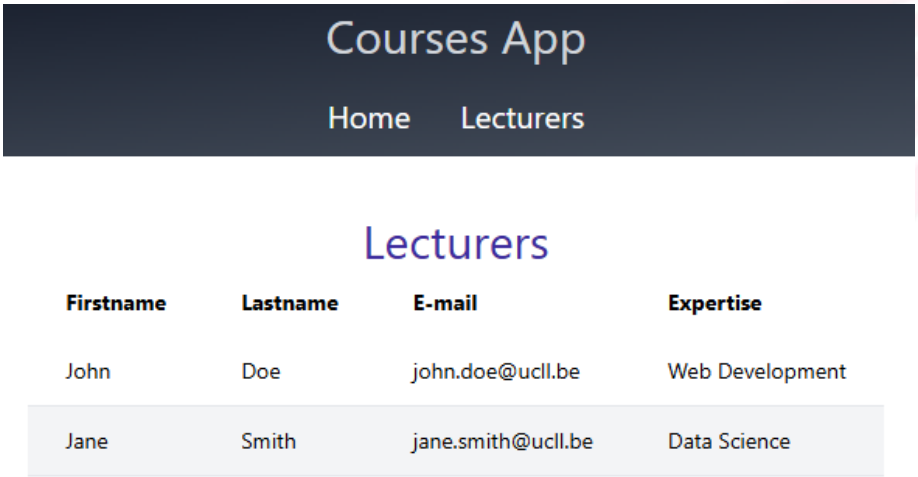
4.3 Make LecturerOverviewTable dynamic

Our table component is still static; it doesn't render the data in the lecturers prop yet.

1) Import the props into the component by destructuring it as a parameter of the component function.
2) Inside the *<tbody>* element, you will need to loop over the *lecturers* parameter to generate a *<tr>* element for each lecturer. The easiest way to do this is with the JavaScript map() function.

```
{lecturers.map((lecturer, index) => (
  <tr key={index} role="button">
     <td>{lecturer.user.firstName}</td>
     <td>{lecturer.user.lastName}</td>
     <td>{lecturer.user.email}</td>
     <td>{lecturer.expertise}</td>
  </tr>
))}
```

3) Go to your browser and verify that the lecturer page looks like this:



4.4 Conditionally render the table

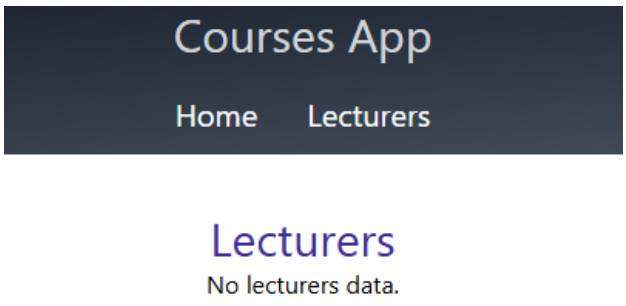
When there is no data (e.g. there are no lecturers in the database), our LecturerOverviewTable doesn't need to be rendered.

1) In lecturers/page.tsx, conditionally render the table component. Remember the inline conditional statement and logical && operator:

```
{ data.length > 0 && ( … )
```

2) When there is no data, only display the message: "No lecturer data.". Add a second inline conditional statement that checks for this scenario (*!data*).

```
{!data.length && <p>No lecturers data.</p>}
```



3) Test both scenarios by toggling the data in *LecturerService* between an empty array and the actual data.

5 Display Courses of the selected Lecturer

When a user clicks on a lecturer (a row) in the table, a second table should appear below it, displaying the courses taught by that lecturer.

5.1 Expand CourseOverviewTable

1) Open the components/CourseOverviewTable component. Similar to what you did for LecturerOverviewTable, make this component dynamic. Start by adding a prop. You can choose whether this component accepts a lecturer object or a courses array.
2) The table should render all courses of the lecturer.
3) The table should only be rendered if the lecturer has courses.

5.2 Conditionally render the CourseOverviewTable

The courses table can only be rendered when a lecturer is selected in the lecturers table. We don't have this information in the overview component, since all logic concerning the lecturers table is encapsulated in the LecturerOverviewTable component. We need a "notification system" so that the table component can notify the overview component when a lecture is selected.

1) To start, add a state field "selectedLecturer" in the lecturer overview page. This state can contain either one Lecturer or null (when no lecturer is selected).
2) We need to pass a callback function to the LecturerOverviewTable component. The component will call this function whenever a lecturer is selected. Extend the props of LecturerOverviewTable so that it accepts a function named selectLecturer:

```
selectLecturer: (lecturer: Lecturer) => void;
```

3) Add an onClick event handler to each table row. This handler should call the selectLecturer function, passing the corresponding lecturer object as an argument. This notifies the parent component that a lecturer has been selected (callback function).
4) In the lecturers overview page, the declaration of LecturerOverviewTable now expects an extra attribute "selectLecturer", to pass the callback function as a prop.

Simply pass the *setSelectedLecturer* state setter function, so that state is immediately updated with the lecturer that was clicked on in the table component.

5) Now render the CourseOverviewTable below the LecturersOverviewTable. Only render this table when *selectedLecturer* contains a value. Pass the *selectedLecturer* (or the lecturers' courses) as a prop to the CourseOverviewTable.
6) Above the table, in the <h2> title with the text "Courses taught by …", replace "…" with the first name of the selected lecturer.
7) Test your changes. The end result should look like this: